

Eksamenforfattere:
Ole Edsberg

Kvalitetskontroll:
Magnus Lie Hetland



Kontakter under eksamen:
Magnus Lie Hetland (918 51 949)
Ole Edsberg (952 81 586)

Kontinuasjonseksamen i
TDT4125 ALGORITMEKONSTRUKSJON, VIDEREGÅENDE KURS

Lørdag 17. august, 2013
Tid: 09:00 – 13:00

Hjelpemidler: (B) Alle trykte/håndskrevne; spesifikk, enkel kalkulator

Språk: Norsk (bokmål)

Les hele eksamen før du begynner, disponer tiden, og forbered spørsmål til fagstabens helperunde. Gjør antakelser der det er nødvendig. Svar kort og konsist. Lange forklaringer som ikke direkte besvarer spørsmålet gis ingen vekt.

Det er 9 oppgaver med til sammen 16 underoppgaver. Alle underoppgavene teller like mye.

Du kan ta for gitt at følgende problemer er NP-harde: TSP, Makespan scheduling, Minimum vertex cover, Minimum set cover, Maximum clique, Maximum cut, Knapsack, Bin-packing, MAX-SAT, IP, 0/1-LP. SAT, 3SAT, Clique, Vertex cover, Hamiltonian cycle.

Hint: Noen av oppgavene er enklere enn de ser ut til.

Oppgave 1

- a) Beskriv hvordan optimeringsproblemet 0/1-LP (binær LP) kan løses med branch-and-bound, uten å benytte LP-relaksering. Beskriv bare følgende elementer: branching, alle bounds du bruker og hvordan du bruker dem, og hvordan du avkutter subtrær.

Løsning: Hver variabel tilsvareer ett nivå i treet. Branching skjer ved å sette neste usatte variabel til 0 eller 1. Pessimistisk bound for optimum er kostnad av beste feasible løsning funnet så langt. Optimistisk bound for optimum i et subtre kan f. eks være kostnad av løsningen hvor alle usatte variabler settes til det alternativet som er mest gunstig for objektivfunksjonen, uten hensyn til constraints (denne er ikke nødvendigvis feasible). Et subtre kuttet hvis optimistisk bound for optimum i subtreet ikke er bedre enn pessimistisk bound for globalt optimum.

- b) Hvordan kan du bruke LP-relaksering og løsning av vanlig (flyttalls) LP som et hjelpemiddel i branch-and-bound for 0/1-LP?

Løsning: LP-relaksering er faktisk veldig nyttig for branch-and-bound på 0/1-LP. Studentene bør kunne tenke seg til noen fornuftige forslag for hvordan LP-relaksering kan utnyttes, men det forventes ikke at de skal gi en omfattende gjennomgang. De bør få full score hvis de kommer med minst ett fornuftig forslag veldig godt forklart, eller minst to fornuftige forslag rimelig godt forklart. Her kommer et svar som overgår det vi forventer av studentene (siden dette var noe de skulle tenke seg til selv, og ikke forklart i pensum). Vi løser én LP-relaksering i hvert subproblem (node) vi har åpnet, før vi brancher. Disse LP-relakseringene har de allerede satt variablene som konstanter (parametre), ikke som variabler. Løsningen av LP-relakseringen for et subproblem kan hjelpe oss på flere måter: (i) Hvis relakseringen ikke har noen feasible løsning kan hele subproblemet avskrives. (ii) Hvis løsningen av relakseringen har heltallig verdi for alle variabler, er denne løsningen feasible og dermed optimal for det urelakserte subproblemet, og vi trenger ikke gå dypere i dette subproblemet, bare sjekke om den nye løsningen er bedre enn beste feasible løsning funnet hittil. (iii) Hvis relakseringen har et optimum vil dette være et optimistisk bound for optimum på det urelakserte subproblemet. (iv) Når vi skal velge hvilket subproblem vi brancher på neste gang, kan det være nyttig å se på kostnaden av optimum for LP-relakseringene til hvert av de allerede åpnete subproblemene. Hvis vi velger subproblemet med best optimistisk bound, vil vi kunne håpe å komme raskere til en god løsning som vil gjøre oss i stand til å kutte ut flere subproblemer. (v) Vi kan også bruke LP-relakseringen i valget av neste variabel vi skal branche et subproblem på. Det kan lønne seg å branche på en variabel som ikke har heltallig verdi i optimum for relakseringen, for da vil vi få mer informasjon i subproblemene vi brancher til. (vi) Ved å sammenligne kostnad av beste feasible løsning funnet hittil med relaksert optimum i rotnoden kan vi finne et upper bound for hvor langt det er igjen til optimum. Dette kan vi bruke til å sette et stopp-kriterium for algoritmen, hvis vi ikke er avhengige av å finne eksakt optimum.

- c) Beskriv hvordan du kan løse 0/1-LP med simulated annealing. Du trenger ikke forklare simulated annealing generelt.

Løsning: Representer løsningen med de samme variablene. Initialisering kan gjøres med tilfeldig tilordning, og transisjoner kan gjøres ved å endre en tilfeldig variabel. For initialisering og transisjon møter man en vanskelighet med at constraints ikke nødvendigvis oppfylles av en hvilken som helst tilordning av variable. Ett alternativ for å håndtere dette er å tillate brutte constraints, men legge på et straffelegg i objektivfunksjonen som gir større straff jo større brudd på constraints. Et annet alternativ er å bruke objektivfunksjonen uendret, men sensurere bort infeasible løsninger, dvs. gjøre tilfeldig tilordning/ending om og om igjen helt til resultatet er feasible.

- d) I simulated annealing, hva er de sannsynlige konsekvensene av å redusere temperaturen (i) for fort, og (ii) for sakte?

Løsning: For fort øker sjansen for å ende opp i dårlig lokalt optimum. For sakte gir unødvendig lang kjøretid.

- e) Hvilke garantier gir henholdsvis branch-and-bound og simulated annealing for optimalitet av løsningen som blir funnet, og hvor lang tid det evt. vil ta å oppnå disse garantiene?

Løsning: Branch-and-bound vil finne optimal løsning, men har i det generelle tilfellet worst-case super-polynomisk kjøretid, altså ingen garanti for å finne optimum i polynomisk tid. Simulated annealing har et asymptotisk konvergens-teorem som sier at sannsynligheten for at algoritmen befinner seg i optimum under visse forutsetninger går mot 1 når antall iterasjoner går mot uendelig, hvilket ikke hjelper oss så mye hvis vi trenger en garantert optimal løsning i endelig tid.

Oppgave 2

- a) Anta at $P \neq NP$. Gjør ett av følgende: bevis at suboptimalitetsproblemet for LP er NP-hardt, eller bevis at det ikke er NP-hardt.

Løsning: (De ble presisert 10:30 at dette handler om suboptimality decision problem, definert på s. 201 i læreboken.) Vi viser at suboptimalitetsproblemet kan løses i polynomisk tid, hvilket betyr at det ikke er NP-hardt med mindre $P = NP$. For å løse suboptimalitetsproblemet må to ting gjøres: sjekke om løsningen er feasible, og sjekke om kostnaden dens er dårligere enn optimum. Førstnevnte krever bare å summere opp venstresiden (lineært i antall variabler) i hver constraint og sammenligne med høyresiden, og sistnevnte kan gjøres ved å sammenligne med det faktiske optimum, som kan finnes med en polynomisk-tid-algoritme for LP (hvilket eksisterer, fordi LP som kjent er løsbart i polynomisk tid).

Oppgave 3

- a) For hvert av følgende tilfeller med informasjon om en algoritme, angi hva vi kan vite om hvorvidt algoritmen er en PTAS, en FPTAS, begge deler, eller ingen av delene. $|x|$ står for inngangsstørrelsen målt i antall bits, og ϵ står for den relative feilen.

1. Algoritmen har worst-case kjøretid i $\Theta(|x|^{\epsilon^{-1}})$.

Løsning: PTAS, ikke FPTAS.

2. Algoritmen har worst-case kjøretid i $O(|x|^{\epsilon^{-1}})$.

Løsning: PTAS og FPTAS

3. Algoritmen har worst-case kjøretid i $O((1 + \epsilon)^{|x|})$.

Løsning: Ukjent.

4. Algoritmen har worst-case kjøretid i $\Theta(|x|)$ og approksimeringsgrad $5/2$.

Løsning: Ingen av delene.

Oppgave 4 I denne oppgaven skal du oppgi alle mulige approksimeringsgrader en algoritme kan gi med følgende grafer som inngang: trekant, firkant, femkant, sekskant. (Disse består av henholdsvis tre, fire, fem og seks noder som er koblet sammen i ring.)

- a) Algoritme 4.3.2.1 (maksimal matching) på s. 262 i læreboka.

Løsning: Trekant: 1, firkant: 2, femkant: $4/3$, sekskant: $4/3$ og $6/3$.

- b) Algoritme 4.3.2.8 (grådig) på s. 264 i læreboka.

Løsning: Trekant: 1, firkant: 1, femkant: 1, sekskant: 1 og $4/3$.

Oppgave 5

- a) Reduser TSP (Traveling Salesperson Problem) til 0/1-LP.

Løsning: Vi bruker en binær variabel $x_{i,j}$ for hvert ordnet par av noder v_i og v_j ($i \neq j$) i grafen, og lar $x_{i,j} = 1$ bety at node v_i blir besøkt umiddelbart før node v_j i TSP-turen. (Merk at vi dermed lar turen ha retning.) Vi må sørge for (1) at alle nodene blir besøkt kun én gang i turen, (2) at vi får kun én tur og ikke flere separate turer, og (3) at kostnaden av turen blir minimert. Vi tar hånd om (1) ved å innføre to constraints (føringer) for hver node v_i i grafen: $\sum_{j \neq i} x_{i,j} = 1$ og $\sum_{j \neq i} x_{j,i} = 1$. For å ta hånd om (2) må vi innføre en heltalls (integer-valued) dummy-variabel y_i for hver node v_i bortsett fra en vilkårlig utvalgt spesialnode v' , og en ny føring for hvert ordnet par av noder (v_i, v_j) , $v_i \neq v_j$, $v_i \neq v'$, $v_j \neq v'$: $y_i - y_j + |V| \cdot x_{i,j} \leq |V| - 1$. Hvis v_i kommer umiddelbart før v_j i turen (med andre ord $x_{i,j} = 1$), krever dette at $y_j \geq y_i$, hvilket gir stigende rekkefølge. Hvis derimot $x_{i,j} = 0$, krever føringen bare at $y_i - y_j \leq |V| - 1$, hvilket f.eks vil være tilfredsstillt hvis dummy-variablene får hver sin verdi fra $1, 2, \dots, |V| - 1$. Objektivfunksjonen (3) blir $\sum_{i,j} x_{i,j} \cdot \text{cost}(\{v_i, v_j\})$. Merk at det opprinnelige løsningsforslaget

ikke tok høyde for problemet med separate turer, jeg (Ole) var dum da jeg skrev det. Det var ikke jeg som sensurerte, men jeg gjorde en sjekk av regnearket fra sensor og konkluderte med at det under rimelige antakelser så ut til at feilen i If heldigvis ikke hadde noen betydning for karakterene, hverken opp eller ned.

Oppgave 6

- a) Anta at $P \neq NP$. La $NumLit$ være en parametrisering av SAT, hvor $NumLit(x)$ for en instans x er lik det største antall literaler per klausul. Gjør ett av følgende: bevis at SAT er fixed-parameter tractable i henhold til $NumLit$, eller bevis at det ikke er fixed-parameter tractable i henhold til $NumLit$.

Løsning: $Set_V(3)$ for $NumLit$ er identisk med 3SAT, som er NP-hardt. Derfor vil en $NumLit$ -parametrisert polynomisk-tid algoritme for SAT løse 3SAT i polynomisk tid, hvilket er umulig med mindre $P = NP$.

Oppgave 7 Det følgende problemet kaller vi *bordplasseringsproblemet*: Du skal plassere n venner ved siden av hverandre rundt et rundt bord, slik at plass 1 er ved siden av plass 2, etc., og plass n er ved siden av plass 1. Hver person sitter altså ved siden av to stykker. Hver person har oppgitt en preferanse (et positivt heltall) for å sitte ved siden av hver av de andre. (Person A trenger ikke ha samme preferanse for person B som person B har for A.) Kvaliteten til en løsning er lik summen av disse preferanseverdiene for de aktuelle bordpartnerne. Målet er å finne den løsningen som har høyest kvalitet.

- a) Anta at $P \neq NP$. Gjør ett av følgende: bevis at bordplasseringsproblemet er sterkt NP-hardt, eller bevis at det ikke er sterkt NP-hardt.

Løsning: Det er sterkt NP-hardt, bevist med reduksjon fra Hamilton-sykel-problemet til terskel-versjonen av bordsettingsproblemet. For å mappe en instans av Hamilton Cycle til bordplasseringsproblemet, la hver person tilsvare en node i grafen, og la preferansen fra en person til en annen (og tilbake) være 2 hvis deres noder er forbundet med en kant, og 1 ellers. Grafen har en Hamilton-sykel hvis og bare hvis optimum for bordplasseringsproblemet er lik $4n$. Implikasjon en retning: Hvis bordplasseringsproblemet har optimum $4n$ (alle liker begge sine bordpartnere med styrke 2) vil det finnes en Hamilton-sykel i grafen som starter med en node/person og så følger bordplasseringen med klokken til sykkelen er sluttet. Implikasjon motsatt retning: Hvis det finnes et Hamilton-sykel i grafen, vil det finnes en bordplassering med kostnad $4n$ hvor nodene/personene sitter i samme rekkefølge som i sykkelen. (Det finnes ikke bordplasseringsløsninger med kostnad høyere enn $4n$, for da er alle preferansene som summeres opp satt til 2, som er høyeste verdi. Mappingen kan utføres i polynomisk tid, for man trenger kun behandle hver node og

hver kant én gang. Vi har redusert et NP-hardt problem til terskel-versjonen av bordsettingsproblemet, uten å bruke andre heltall enn 1 og 2 (altså polynomisk bundet i problemstørrelsen). Altså har vi bevist at bordsettingsproblemet er sterkt NP-hardt.

Oppgave 8 Du skal kjøpe inn k ryggsekker til en ekspedisjon med k medlemmer som bærer én ryggsekk hver. Ekspedisjonen skal til sammen bære med seg n gjenstander, som hver har en vekt (et flyttall større enn 0). En gjenstand kan ikke deles på flere ryggsekker. Forskjellige typer ryggsekker har forskjellig kapasitet for hvor stor vekt de tåler. Fordi du er en ordensfrik, må alle ryggsekkene være identiske. Du ønsker å vite hvor lav kapasitet per ryggsekk ekspedisjonen kan klare seg med, og likevel få med alle gjenstandene.

- a) Gi en approksimeringsalgoritme som løser problemet i polynomisk tid, med approksimeringsgrad $7/3$ eller bedre. (Her finnes det noe du kan bruke i læreboka, men du må likevel beskrive algoritmen med egne ord.)

Løsning: Dette er nesten MAKESPAN-scheduling, som har en 2-approksimeringsalgoritme på s. 250 i læreboka. Eneste forskjell er at vektene i ryggsekk-problemet har flyttall, hvilket ikke har noen betydning. Til sensor: det er mulig, og bør ikke trekkes for, å formulere seg enklere enn læreboka.

- b) Gi et bevis for approksimeringsgraden til algoritmen din. (Her finnes det noe du kan bruke i læreboka, men du må likevel gi beviset i egne ord.)

Løsning: Se s.250-251 i læreboka. Igjen er det mulig, og bør ikke trekkes for, å formulere seg enklere enn læreboka.

Oppgave 9 Du har fått i oppdrag å levere trådløst internett til innbyggerne i et asteroidfelt. Du har til disposisjon k sender/mottaker-stasjoner. En sender/mottaker-stasjon kan monteres på en asteroide. Den har en langdistanse-forbindelse til Tellus (jorden), og tilbyr trådløst internett en viss avstand ut i alle retninger fra asteroiden den er montert på, med synkende kvalitet lenger unna. Du kan anta at asteroidene kan modelleres som punkter med stabile relative posisjoner i et tre-dimensjonalt koordinatsystem, og at de ikke blokkerer trådløs-signalet på noen som helst måte. Ditt mål er å velge k asteroider som skal få plassert sender/mottaker-stasjon, slik at den lengste avstanden fra en asteroide uten stasjon til en asteroide med stasjon minimeres. (Oppgave basert på Williamson & Shmoys.)

- a) Gi en approksimeringsalgoritme som løser problemet i polynomisk tid. For å få full score må algoritmen din ha approksimeringsgrad 2 eller bedre, men du vil få en viss uttelling for fornuftige forsøk med dårligere approksimeringsgrad. (Det finnes en enkel løsning basert på en av algoritmedesignteknikkene i kapittel 2.3.4.)

Løsning: (Grådig.) Velg den første asteroiden vilkårlig. Genta $k - 1$ ganger: velg den

asteroiden som maksimerer avstanden til den nærmeste av de allerede valgte asteroidene. (Oppgave basert på Williamson & Shmoys.)

b) Gi et bevis for approksimeringsgraden til algoritmen din.

Løsning: La r være verdien av objektivfunksjonen (lengste avstand fra asteroide uten stasjon til asteroide med stasjon) i den optimale løsningen. La s være verdien av objektivfunksjonen i løsningen funnet av algoritmen vår. Vi skal bevise at det alltid vil gjelde at $s \leq 2r$. Våre valgte stasjon-asteroider kan fordele seg på to måter i sammenheng med de optimale stasjon-asteroidene.

1. Alle våre stasjon-asteroider ligger nærmest en forskjellig optimal stasjon-asteroide. Det betyr at avstanden fra en av våre stasjon-asteroider til nærmeste optimale stasjon-asteroide er maksimalt r . Når vi også vet per definisjon at ingen asteroider ligger lenger vekk fra en optimal stasjon-asteroide enn r , er det klart at den avstanden fra en hvilken som helst asteroide til en av våre stasjon-asteroider ikke kan være større enn $2r$.
2. To eller flere av våre stasjon-asteroider har samme nærmeste optimale stasjon-asteroider. Dette kan bare skje hvis algoritmen vår i en eller annen iterasjon velger en stasjon-asteroide u med samme nærmeste stasjon-asteroide som en allerede valgt stasjon-asteroide v . Fordi u og v har samme nærmeste stasjon-asteroide, kan ikke avstanden mellom dem være større enn $2r$. Men u ble valgt fordi u var den asteroiden med lengst avstand til en allerede valgt stasjon-asteroide. Derfor finnes det ingen asteroide med større avstand til den nærmeste av våre stasjon-asteroider enn $2r$. Dette gjelder for alle iterasjoner av algoritmen etter at vi har valgt to asteroider med samme nærmeste optimale stasjon-asteroide.

(Oppgave basert på Williamson & Shmoys.)